

- Code - fixed size $\Rightarrow$ memory requirement statically determined
- Data - global data objects can be statically allocated
- Stack - area used to keep track of control stack / activations
- Heap - all other data (esp. memory (de)allocated under program control via malloc, free, etc.)
- Stack and Heap grow in opposite directions

Program organization

- Program is made up of procedures
- Procedure = name + parameters + local variables + return value + body
- Procedure call $\Rightarrow$ body of the procedure is executed; control returns to instruction immediately following procedure call

Activation: each execution of a procedure

Lifetime: sequence of steps between 1st and last step in the execution of procedure body (including calls to other procedures from within the body)

Lifetimes of procedure activations are either non-overlapping or nested

Activation tree: graphical representation of flow of activations when a program is running

1. Node $\equiv$ activation of a procedure (root $\equiv$ “main”)
2. Node a is parent of $b \Leftrightarrow$ control flows from activation a to b
3. For two siblings a, b : a is to the left of $b \Leftrightarrow$ lifetime of a is before lifetime of b

Actual flow of control corresponds to depth-first traversal of tree

- Temporary variables (compiler created)
- Local data, variables
- Saved machine status (PC, other register values)
- Access link (optional) - specifies where non-local data can be found
- Control link (optional) - points to activation record of caller
- Actual parameters (registers may also be used)
- Return value

Static allocation:

- Names are bound to storage locations at compile time
⇒ a name is always bound to the same location
- All activation records for a procedure are stored at the same location

Disadvantages:

1. Data size must be known at compile time (no dynamic allocation / free-ing of memory)
2. No recursion is possible

Scheme: Stack (*control stack*) is used to keep track of live activations

- use a register (SP) to point to AR at top of control stack
- push AR on stack when procedure is called; pop AR when procedure returns
 - ⇒ locals bound to fresh storage in each activation; values of locals lost when activation ends

Layout of AR:

- fixed size fields should be placed in initial part of the AR
- temporaries and local data should be stored last
- parameters and return value should also be stored in the initial part of the AR, immediately next to the caller's AR

Calling sequence: code executed when procedure is called

Return sequence: code executed when procedure returns

Calling/return sequence

- Preferable to put most of the calling / return sequence in callee

Calling sequence:

1. Caller: evaluates arguments
2. Caller: stores PC, SP; increments SP ($\equiv$ creating / pushing AR on top of stack)
3. Callee: save values of other registers, status information
4. Callee: initialize local data and begin execution

Return sequence:

1. Callee: stores return value next to caller's AR
2. Callee: restores SP, registers, and branches to saved PC
3. Caller: retrieves return value if necessary

Variable length parameter lists

- Callee code must handle variable length parameter lists
- Usually, first parameter specifies number of parameters

Example: printf

Dangling references: reference to deallocated storage

1. Call by value
2. Call by reference
3. Copy restore / call by value-result
4. Call by name

Block: statement containing its own local data declarations

- Blocks are nested or non-overlapping
- *Most-closely nested* scope rules
- Memory allocation method - I
 - Allocate storage for complete procedure body at one time (including storage for all blocks in the procedure)
- Memory allocation method - II
 - Treat a block as a parameterless procedure
 - Block variables $\equiv$ local vars. for procedures
 - Block variables allocated when block is entered; deallocated on leaving block
 - Flow of control is simple: called only from point just before block and returns to point just after block

NOTE: jumps into and out of block would create problems

Access link: if procedure P is nested immediately within Q in the source text, then the access link in AR_P points to the access link in the AR for the most recent activation for Q

Languages with no nested procedures:

- If a is non-local, then it must be global
- All global variables are allocated statically (addresses determined at compile time)
- No access link needed

Languages with nested procedures:

Nesting depth (ND):

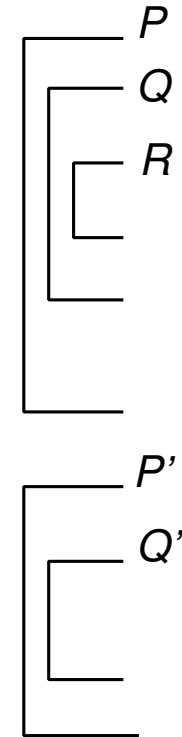
- ND of top-level procedure (e.g. main) is 1
- if P is nested immediately within Q , $ND_P = ND_Q + 1$

Setting up access links:

Let procedure P (nesting depth N_p) call Q (nesting depth N_q).

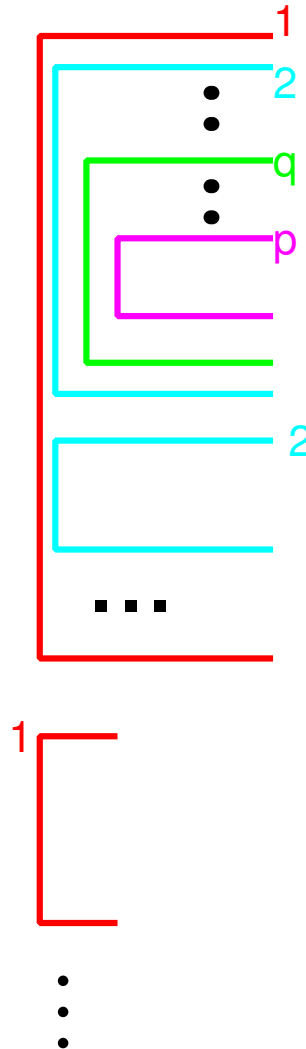
1. If $N_p < N_q$:

- Q must be declared within P
- make access link in AR_Q point to AR_P
(located just below AR_Q in stack)



2. If $N_p \geq N_q$:

- enclosing procedures at depths $1 \dots N_q - 1$ must be same for both P and Q
- follow $N_p - N_q + 1$ access links from caller (P) to reach the most recent AR of the procedure that statically encloses both P and Q most closely
- set access link in AR_Q to point to this AR



Using access links:

Suppose we have a reference to non-local variable a in procedure p . To find storage location for a :

1. Follow $N_p - N_a$ links from the AR at top of stack (AR_p).
2. The AR reached corresponds to the procedure that a is local to.
3. Storage location of a is at a known offset within the *local variables - temps* section of this AR